

Unit-2: Fundamentals of React.js

2.1 Overview of React

What Is React?

React.js is a front-end JavaScript framework developed by Facebook. To build composable user interfaces predictably and efficiently using declarative code, we use React. It's an open-source and component-based framework responsible for creating the application's view layer.

- React.js follows the Model View Controller (MVC) architecture, and the view layer is accountable for handling mobile and web apps.
- React is famous for building single-page applications and mobile apps.
- As the figure shows, React.js divides the UI into multiple components, making the code easier to debug.
- This way, each function is assigned to a specific component, and it produces some HTML which is rendered as output by the DOM.



How does React Work?

- React creates a VIRTUAL DOM in memory.
- Instead of manipulating the browser's DOM directly, React creates a virtual DOM in memory, where it does all the necessary manipulating, before making the changes in the browser DOM.
- React only changes what needs to be changed!
- React finds out what changes have been made, and changes only what needs to be changed.
- You will learn the various aspects of how React does this in the rest of this tutorial.

ReactJS History

- Facebook Software Engineer, Jordan Walke, created it.
- React.JS was first used in 2011 for Facebook's Newsfeed feature.
- Initial Release to the Public (V0.3.0) was in July 2013.
- Current version of create-react-app is v18.2.0 (14 June 2022).
- create-react-app includes built tools such as webpack, Babel, and ESLint.

Why React?

React's popularity today has eclipsed that of all other front-end development frameworks.

- **Easy creation of dynamic applications:** React makes it easier to create dynamic web applications because it requires less coding and offers more functionality, as opposed to JavaScript, where coding often gets complex very quickly.
- **Improved performance:** React uses Virtual DOM, thereby creating web applications faster. Virtual DOM compares the components' previous states and updates only the items in the Real DOM that were changed, instead of updating all of the components again, as conventional web applications do.
- **Reusable components:** Components are the building blocks of any React application, and a single app usually consists of multiple components. These components have their logic and controls, and they can be reused throughout the application, which in turn dramatically reduces the application's development time.
- **Unidirectional data flow:** React follows a unidirectional data flow. This means that when designing a React app, developers often nest child components within parent components. Since the data flows in a single direction, it becomes easier to debug errors and know where a problem occurs in an application at the moment in question.

- **Small learning curve:** React is easy to learn, as it mostly combines basic HTML and JavaScript concepts with some beneficial additions. Still, as is the case with other tools and frameworks, you have to spend some time to get a proper understanding of React's library.
- **It can be used for the development of both web and mobile apps:** We already know that React is used for the development of web applications, but that's not all it can do. There is a framework called React Native, derived from React itself, that is hugely popular and is used for creating beautiful mobile applications. So, in reality, React can be used for making both web and mobile applications.
- **Dedicated tools for easy debugging:** Facebook has released a Chrome extension that can be used to debug React applications. This makes the process of debugging React web applications faster and easier.

Features

The salient features of React library are as follows –

- Solid base architecture
- Extensible architecture
- Component based library
- JSX based design architecture
- Declarative UI library

Benefits

Few benefits of using React library are as follows –

- Easy to learn
- Easy to adept in modern as well as legacy application
- Faster way to code a functionality
- Availability of large number of ready-made component
- Large and active community

Applications

Few popular websites powered by React library are listed below –

- Facebook
- Instagram
- Netflix
- Code Academy
- Reddit

- Airbnb

ReactJS - Installation

1. Install vs code or any editor
2. Create a folder to save a program in any directory
3. Open that folder in cmd
4. You have installed Node.js on your machine.
5. We can check it using the below command

node --version

6. You could see the version of Nodejs you might have installed.

v14.2.0

7. If not installed, you can download and install by visiting

<https://nodejs.org/en/download/>.

8. Check node installed or not

node --version

9. Check npm install or not

npm --version

10. Now create app

npx create-react-app ReactExample

11. After completion process, Jump in to the ReactExample folder.

cd ReactExample

12. To open your program in vs code type

code .

It will open your code in vs code

13. Now start your project

npm start

14. Now check in browser your app will be started. At 3000 port

Create basic program to display hello world**App.js**

```
import React from "react";
import ReactDOM from "react-dom/client";

function Hello(props) {
  return <h1>Hello World!</h1>;
}

const container = document.getElementById('root');
const root = ReactDOM.createRoot(container);
root.render(<Hello />);
```

2.1.2. Using react with HTML

- React JSX is an extension to JavaScript.
- JSX allows us to write HTML in React.
- JSX makes it easier to write and add HTML in React.
- It enables developer to create virtual DOM using XML syntax.
- It compiles down to pure JavaScript (*React.createElement function calls*).
- JSX converts HTML tags into react elements.
- JSX provides you to write HTML/XML-like structures (e.g., DOM-like tree structures) in the same file where you write JavaScript code, and then pre-processor will transform these expressions into actual JavaScript code. Just like XML/HTML, JSX tags have a tag name, attributes, and children. Components are like functions that return HTML elements. Components are independent and reusable bits of code. They serve the same purpose as JavaScript functions, but work in isolation and return HTML.
- React renders HTML to the web page by using a function called **ReactDOM.render()**.
- Since it compiles to JavaScript, it can be used inside any valid JavaScript code. For example, below codes are perfectly valid.
- Assign to a variable.

```
var greeting = <h1>Hello React!</h1>
```

- Assign to a variable based on a condition.

```
var canGreet = true;
if(canGreet) {
  greeting = <h1>Hello React!</h1>
```

```
}
```

- Can be used as return value of a function.

```
function Greeting() {
  return <h1>Hello React!</h1>
}

greeting = Greeting()
```

- Can be used as argument of a function.

```
function Greet(message) {
  ReactDOM.render(message, document.getElementById('react-app'))
}

Greet(<h1>Hello React!</h1>)
```

Expressions

JSX supports expression in pure JavaScript syntax. Expression has to be enclosed inside the curly braces, { }. Expression can contain all variables available in the context, where the JSX is defined. Let us create simple JSX with expression.

Example:

```
<script type="text/babel">
  var cTime = new Date().toLocaleTimeString();
  ReactDOM.render(
    <div><p>The current time is {cTime}</p></div>,
    document.getElementById('react-app') );
</script>
```

Output:

The Current time is 21:19:56 GMT+0530(India Standard Time)

Functions

JSX supports user defined JavaScript function. Function usage is similar to expression. Let us create a simple function and use it inside JSX.

Example

```
<script type="text/babel">
  var cTime = new Date().toLocaleTimeString();
  ReactDOM.render(
    <div><p>The current time is {cTime}</p></div>,
    document.getElementById('react-app')
```

```
);  
</script>
```

Output

The Current time is 21:19:56 GMT+0530(India Standard Time)

Attributes

JSX supports HTML like attributes. All HTML tags and its attributes are supported. Attributes has to be specified using camelCase convention (and it follows JavaScript DOM API) instead of normal HTML attribute name. For example, class attribute in HTML has to be defined as *className*. The following are few other examples –

- *htmlFor* instead of *for*
- *tabIndex* instead of *tabindex*
- *onClick* instead of *onclick*

Example

```
<style>  
  .red { color: red }  
</style>  
<script type="text/babel">  
  function getCurrentTime() {  
    return new Date().toString();  
  }  
  ReactDOM.render(  
    <div>  
      <p>The current time is <span  
className="red">{getCurrentTime()}</span></p>  
    </div>,  
    document.getElementById('react-app')  
  );  
</script>
```

Output

The Current time is 22:36:55 GMT+0530(India Standard Time)

Expression in attributes

JSX supports expression to be specified inside the attributes. In attributes, double quote should not be used along with expression. Either expression or string using double quote has to be used. The above example can be changed to use expression in attributes.

```
<style>
```

By: Pooja Soni

```
.red { color: red }
</style>

<script type="text/babel">
  function getCurrentTime() {
    return new Date().toString();
  }
  var class_name = "red";
  ReactDOM.render(
    <div>
      <p>The current time is <span
className={class_name}>{getCurrentTime()}</span></p>
    </div>,
    document.getElementById('react-app')
  );
</script>
```

2.1.3 React Interactive Components: Components within Components & Files

ReactJS - Component

React component is the building block of a React application. Let us learn how to create a new React component and the features of React components in this chapter.

A React component represents a small chunk of user interface in a webpage. The primary job of a React component is to render its user interface and update it whenever its internal state is changed. In addition to rendering the UI, it manages the events belongs to its user interface. To summarize, React component provides below functionalities.

- Initial rendering of the user interface.
- Management and handling of events.
- Updating the user interface whenever the internal state is changed.

Creating a React component

React library has two component types. The types are categorized based on the way it is being created.

- Function component – Uses plain JavaScript function.
- ES6 class component – Uses ES6 class.

The core difference between function and class component are –

- Function components are very minimal in nature. Its only requirement is to return a *React element*.

Components

In React, function components are a way to write components that only contain a render method and don't have their own state. They are simply JavaScript functions that may or may not receive data as parameters. We can create a function that takes props(properties) as input and returns what should be rendered. A valid functional component can be shown in the below example.

```
function WelcomeMessage(props) {  
  return <h1>Welcome to the , {props.name}</h1>;  
}
```

The functional component is also known as a stateless component because they do not hold or manage state. It can be explained in the below example.

Example: App.js

```
import React, { Component } from 'react';  
  
class App extends React.Component {  
  render() {  
    return (  
      <div> <First/> <Second/> </div>  
    );  
  }  
}  
  
class First extends React.Component {  
  render() {  
    return (  
      <div>  
        <h1>React JS</h1>  
      </div>  
    );  
  }  
}  
  
class Second extends React.Component {  
  render() {  
    return (  
      <div>  
        <h1>React JS</h1>  
      </div>  
    );  
  }  
}
```

```
    <div>
      <h2>Well Come To AWD World </h2>
      <p>This is your class .</p>
    </div>
  );
}
}
export default App;
```

Output

React JS

Well Come To AWD World

This is your class

Class Components:

The class components are a little more complex than the functional components. The functional components are not aware of the other components in your program whereas the class components can work with each other. We can pass data from one class component to another class component. We can use JavaScript ES6 classes to create class-based components in React. The below example shows a valid class-based component in React:

```
class Democomponent extends React.Component {
  render() {
    return <h1>Welcome Message!</h1>;
  }
}
```

Example: App.js

```
import React, { Component } from 'react';
class App extends React.Component {
  constructor() {
    super();
    this.state = {
      data:
      [
        {
```

```
        "name": "Rajat"
      },
      {
        "name": "Ajay"
      },
      {
        "name": "Rohit"
      }
    ]
  }
}
render() {
  return (
    <div>
      <StudentName />
      <ul>
        {this.state.data.map((item) => <List data = {item} />)}
      </ul>
    </div>
  );
}
}
class StudentName extends React.Component {
  render() {
    return (
      <div>
        <h1>Employee Name Detail</h1>
      </div>
    );
  }
}
class List extends React.Component {
  render() {
    return (
      <ul>
        <li>{this.props.data.name}</li>
      </ul>
    );
  }
}
export default App;
```

Output:

Employee Name Detail

- Rajat
- Ajay
- Rohit

Rendering a Component

Now your React application has a component called Car, which returns an `<h2>` element.

To use this component in your application, use similar syntax as normal HTML: `<Car />`

Example: Display the Car component in the "root" element:

```
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(<Car />);
```

Hi, I am a Car!

Props

- Components can be passed as **props**, which stands for properties.
- Props are like function arguments, and you send them into the component as attributes.

Example: Use an attribute to pass a color to the Car component, and use it in the `render()` function:

```
function Car(props) {  
  return <h2>I am a {props.color} Car!</h2>;  
}  
  
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(<Car color="red"/>);
```

Output:

I am a red Car!

Components in Components

We can refer to components inside other components:

Example: Use the Car component inside the Garage component:

```
function Car() {  
  return <h2>I am a Car!</h2>;  
}  
  
function Garage() {  
  return (  
    <>  
    <h1>Who lives in my Garage?</h1>  
    <Car />  
  </>  
);  
}  
  
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(<Garage />);
```

Output:

Who lives in my Garage?

I am a Car!

Components in Files

React is all about re-using code, and it is recommended to split your components into separate files.

To do that, create a new file with a **.js** file extension and put the code inside it:

Example: This is the new file, we named it "Car.js":

```
function Car() {  
  return <h2>Hi, I am a Car!</h2>;  
}  
  
export default Car;
```

To be able to use the Car component, you have to import the file in your application.

By: Pooja Soni

Exmple:

```
import React from 'react';  
import ReactDOM from 'react-dom/client';  
import Car from './Car.js';  
  
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(<Car />);
```

Output:

Hi, I am a Car!

Functional Components vs Class Components:

<u>Functional Components</u>	<u>Class Components</u>
A functional component is just a plain JavaScript pure function that accepts props as an argument and returns a React element(JSX).	A class component requires you to extend from React. Component and create a render function that returns a React element.
There is no render method used in functional components.	It must have the render() method returning JSX (which is syntactically similar to HTML)
Functional components run from top to bottom and once the function is returned it can't be kept alive.	The class component is instantiated and different life cycle method is kept alive and is run and invoked depending on the phase of the class component.
Also known as Stateless components as they simply accept data and display them in	Also known as Stateful components because they implement logic and state.

some form, they are mainly responsible for rendering UI.	
React lifecycle methods (for example, componentDidMount) cannot be used in functional components.	React lifecycle methods can be used inside class components (for example, componentDidMount).
Hooks can be easily used in functional components to make them Stateful. Example: <pre>const [name,SetName]= React.useState(' ')</pre>	It requires different syntax inside a class component to implement hooks. Example: <pre>constructor(props) { super(props); this.state = {name: ' '} }</pre>
Constructors are not used.	Constructor is used as it needs to store state.

React Props

Props stand for "Properties." They are read-only components. It is an object which stores the value of attributes of a tag and work similar to the HTML attributes. It gives a way to pass data from one component to other components. It is similar to function arguments. Props are passed to the component in the same way as arguments passed in a function.

Props are immutable so we cannot modify the props from inside the component. Inside the components, we can add attributes called props. These attributes are available in the component as **this.props** and can be used to render dynamic data in our render method.

When you need immutable data in the component, you have to add props to **ReactDOM.render()** method in the main.js file of your ReactJS project and used it inside the component in which you need. It can be explained in the below example.

Example: App.js

```
import React, { Component } from 'react';

class App extends React.Component {
  render() {
    return (
```

```
<div> <h1> Welcome to { this.props.name } </h1>
  <p> <h4> React.js is a front-end JavaScript framework developed by Face book. </h4> </p>
</div>

);
}
}
export default App
```

Main.js

```
import React from 'react';
import ReactDOM from 'react-dom';
import App from './App.js';

ReactDOM.render(<App name = "React!!" />, document.getElementById('app'));
```

Output:

Welcome to React

React.js is a front-end JavaScript framework developed by Face book.

Default Props

It is not necessary to always add props in the ReactDOM.render() element. You can also set **default** props directly on the component constructor. It can be explained in the below example.

Example: App.js

```
import React, { Component } from 'react';
class App extends React.Component {
  render() {
    return (
      <div>
        <h1>Default Props Example</h1>
        <h3>Welcome to {this.props.name}</h3>
      </div>
    )
  }
}
```



```
        <p> React.js is a front-end JavaScript framework developed by Face book.
    </p>
    </div>
    );
  }
}
App.defaultProps = {
  name: " React"
}
export default App;
```

Main.js

```
import React from 'react';
import ReactDOM from 'react-dom';
import App from './App.js';
```

```
ReactDOM.render(<App/>, document.getElementById('app'));
```

Output

Default Props Example

Welcome to React

React.js is a front-end JavaScript framework developed by Face book.

2.2.2 Conditional statements, Operators, Lists

In React, we can create multiple components which encapsulate behavior that we need. After that, we can render them depending on some conditions or the state of our application. In other words, based on one or several conditions, a component decides which elements it will return. In React, conditional rendering works the same way as the conditions work in JavaScript. We use JavaScript operators to create elements representing the current state, and then React Component update the UI to match them.

There is more than one way to do conditional rendering in React. They are given below.

- If statement

- ternary operator
- logical && operator
- switch case operator
- Conditional Rendering with enums

1. If statement

It is the easiest way to have a conditional rendering in React in the render method. It is restricted to the total block of the component. IF the condition is **true**, it will return the element to be rendered. It can be understood in the below example.

Example

```
function UserLogin(props) {  
  return <h1>Welcome back!</h1>;  
}  
  
function GuestLogin(props) {  
  return <h1>Please sign up.</h1>;  
}  
  
function SignUp(props) {  
  const isLoggedIn = props.isLoggedIn;  
  if (isLoggedIn) {  
    return <UserLogin />;  
  }  
  return <GuestLogin />;  
}  
  
ReactDOM.render(  
  <SignUp isLoggedIn={false} />,  
  document.getElementById('root')  
);
```

Logical && operator

By: Pooja Soni

This operator is used for checking the condition. If the condition is **true**, it will return the element **right** after **&&**, and if it is **false**, React will **ignore** and skip it.

Syntax

```
{  
  condition &&  
  // whatever written after && will be a part of output.  
}
```

We can understand the behavior of this concept from the below example.

If you run the below code, you will not see the **alert** message because the condition is not matching.

```
('react' == 'React') && alert('This alert will never be shown!')
```

If you run the below code, you will see the **alert** message because the condition is matching. `(10 > 5) && alert('This alert will be shown!')`

Example:

```
import React from 'react';  
import ReactDOM from 'react-dom';  
  
// Example Component  
function Example()  
{  
  return(<div>  
    {  
      (10 > 5) && alert('This alert will be shown!')  
    }  
    </div>  
  );  
}
```

```
}
```

You can see in the above output that as the condition **(10 > 5)** evaluates to true, the alert message is successfully rendered on the screen.

Ternary operator

The ternary operator is used in cases where two blocks alternate given a certain condition. This operator makes your if-else statement more concise. It takes **three** operands and used as a shortcut for the if statement.

Syntax

```
condition ? true : false
```

If the condition is **true**, **statement1** will be rendered. Otherwise, **false** will be rendered.

Example:

```
render() {  
  
  const isLoggedIn = this.state.isLoggedIn;  
  
  return (  
    <div>  
      Welcome {isLoggedIn ? 'Back' : 'Please login first'}.  
    </div>  
  );  
}
```

React Lists

Lists are used to display data in an ordered format and mainly used to display menus on websites. In React, Lists can be created in a similar way as we create lists in JavaScript. Let us see how we transform Lists in regular JavaScript.

The `map()` function is used for traversing the lists. In the below example, the `map()` function takes an array of numbers and multiply their values with 5. We assign the new array returned by `map()` to the variable `multiply Nums` and log it.

Example :

```
var numbers = [1, 2, 3, 4, 5];

const multiplyNums = numbers.map((number)=>{

  return (number * 5);

});

console.log(multiplyNums);
```

Output

[5, 10, 15, 20, 25]

Example:

```
import React from 'react';
import ReactDOM from 'react-dom';

const myList = ['Payal', 'Sachin', 'Kevin', 'Disha', 'Asha'];
const listItems = myList.map((myList)=>{
  return <li>{myList}</li>;
});
ReactDOM.render(
  <ul> {listItems} </ul>,
  document.getElementById('app')
);
export default App;
```

Output

- Payal
- Sachin
- Kevin
- Disha
- Asha

React Keys

A key is a unique identifier. In React, it is used to identify which items have changed, updated, or deleted from the Lists. It is useful when we dynamically created components or when the

users alter the lists. It also helps to determine which components in a collection needs to be re-rendered instead of re-rendering the entire set of components every time.

Keys should be given inside the array to give the elements a stable identity. The best way to pick a key as a string that uniquely identifies the items in the list. It can be understood with the below example.

Example:

```
const stringLists = ['Payal', 'Sachin', 'Kevin', 'Disha', 'Asha' ];

const updatedLists = stringLists.map((strList)=>{

  <li key={strList.id}> {strList} </li>;

});
```

If there are no stable IDs for rendered items, you can assign the item **index** as a key to the lists. It can be shown in the below example.

Example:

```
const stringLists = [ 'Peter', 'Sachin', 'Kevin', 'Dhoni', 'Alisa' ];

const updatedLists = stringLists.map((strList, index)=>{

  <li key={index}> {strList} </li>;

});
```

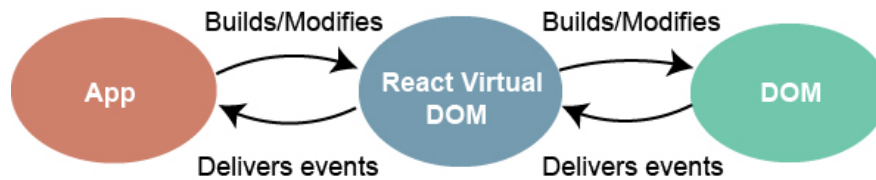
2.2.3 React Events: Adding events, Passing arguments, Event objects

React Events

An event is an action that could be triggered as a result of the user action or system generated event. For example, a mouse click, loading of a web page, pressing a key, window resizes, and other interactions are called events.

React has its own event handling system which is very similar to handling events on DOM elements. The react event handling system is known as Synthetic Events. The synthetic event is a cross-browser wrapper of the browser's native event.

Events Handler



Handling events with react have some syntactic differences from handling events on DOM. These are:

1. React events are named as **camelCase** instead of **lowercase**. `onClick` instead of `onclick`.
2. With JSX, a function is passed as the **event handler** instead of a **string**. For example:

```
<button onclick="showMessage()">
  Hello ReactJS
</button>
```

Example:

```
import React from 'react';
import ReactDOM from 'react-dom/client';

function Football() {
  const shoot = () => {
    alert("Great Shot!");
  }

  return (
    <button onClick={shoot}>Take the shot!</button>
  );
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Football />);
```

Passing Arguments

To pass an argument to an event handler, use an arrow function.

Example:

By: Pooja Soni

```
import React from 'react';
import ReactDOM from 'react-dom/client';

function Football() {
  const shoot = (a) => {
    alert(a);
  }

  return (
    <button onClick={() => shoot("Goal!")}>Take the shot!</button>
  );
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Football />);
```

React Event Object

Event handlers have access to the React event that triggered the function.

In our example the event is the "click" event.

Example:

```
import React from 'react';
import ReactDOM from 'react-dom/client';

function Football() {
  const shoot = (a, b) => {
    alert(b.type);
    /*
      'b' represents the React event that triggered the function.
      In this case, the 'click' event
    */
  }

  return (
    <button onClick={(event) => shoot("Goal!", event)}>Take the shot!</button>
  );
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Football />);
```


Example

In the below example, we have used only one component and adding an onChange event. This event will trigger the **changeText** function, which returns the company name.

```
import React, { Component } from 'react';
class App extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      companyName: ""
    };
  }
  changeText(event) {
    this.setState({
      companyName: event.target.value
    });
  }
  render() {
    return (
      <div>
        <h2>Simple Event Example</h2>
        <label htmlFor="name">Enter company name: </label>
        <input type="text" id="companyName" onChange={this.changeText.bind(this)} />

        <h4>You entered: { this.state.companyName }</h4>
      </div>
    );
  }
}
export default App;
```

Angular JS V/S React JS

	AngularJS	ReactJS
Author	Google	Facebook Community
Developer	Misko Hevery	Jordan Walke
Initial Release	October 2010	March 2013
Latest Version	Angular 1.7.8 on 11 March 2019.	React 16.8.6 on 27 March 2019
Language	JavaScript, HTML	JSX
Type	Open Source MVC Framework	Open Source JS Framework
Rendering	Client-Side	Server-Side
Packaging	Weak	Strong
Data-Binding	Bi-directional	Uni-directional
DOM	Regular DOM	Virtual DOM
Testing	Unit and Integration Testing	Unit Testing
App Architecture	MVC	Flux
Dependencies	It manages dependencies automatically.	It requires additional tools to manage dependencies.
Routing	It requires a template or controller to its router configuration, which has to be managed manually.	It doesn't handle routing but has a lot of modules for routing, eg., react-router.
Performance	Slow	Fast, due to virtual DOM.
Best For	It is best for single page applications that update a single view at a time.	It is best for single page applications that update multiple views at a time.